셰이더 프로그래밍
LECTURE5

2026년 1학기

담당교수 : 게임공학과 이택희

수업에서 다룰 내용

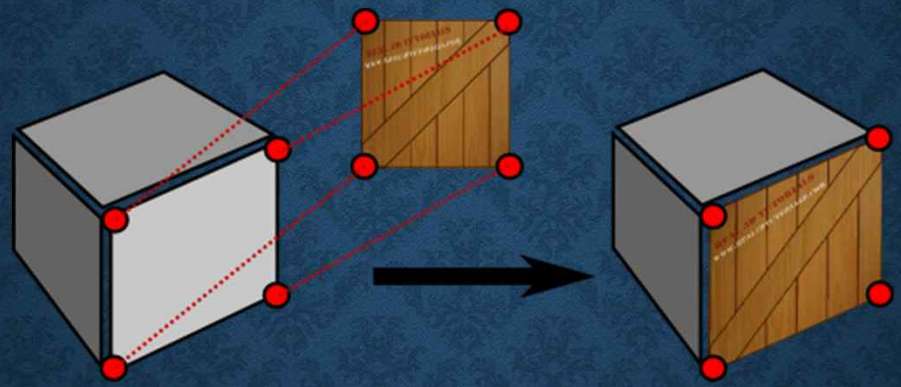
- 텍스쳐 매핑 Vertex -> x,y,z tx, ty
  u v
- 텍스쳐 생성 png -> texture (병렬 process)
  "CUDA"
- 텍스쳐 사용
- 다중 텍스처 사용
- 단일 텍스처 사용 애니메이션

텍스쳐 매핑

텍스쳐 매핑

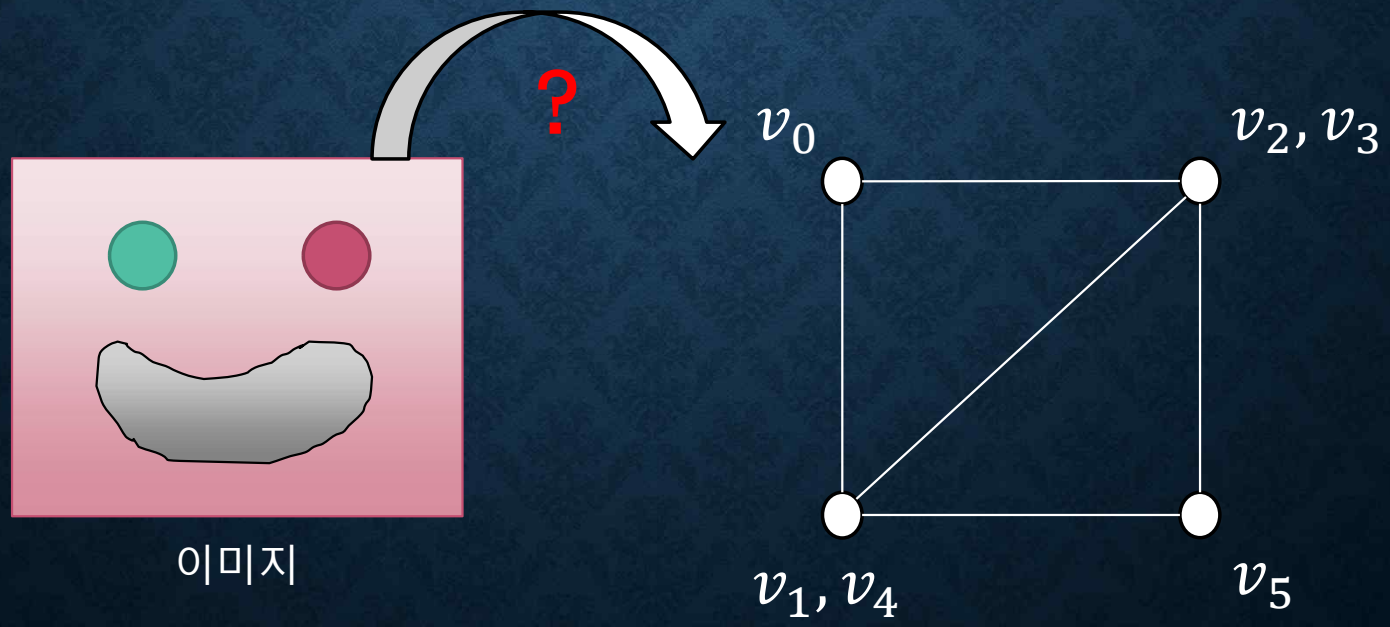
- 텍스쳐란?
  - 물체의 표면의 디테일을 표현하고자 할 때 모두 버텍스로 표현하기 위해선 매우 많은 데이터가 필요
  - 해당 부분을 이미지로 대체 할 경우 간단하게 복잡한 표면을 표현 할 수 있음
  - 이 이미지를 텍스쳐라 함

텍스쳐 매핑



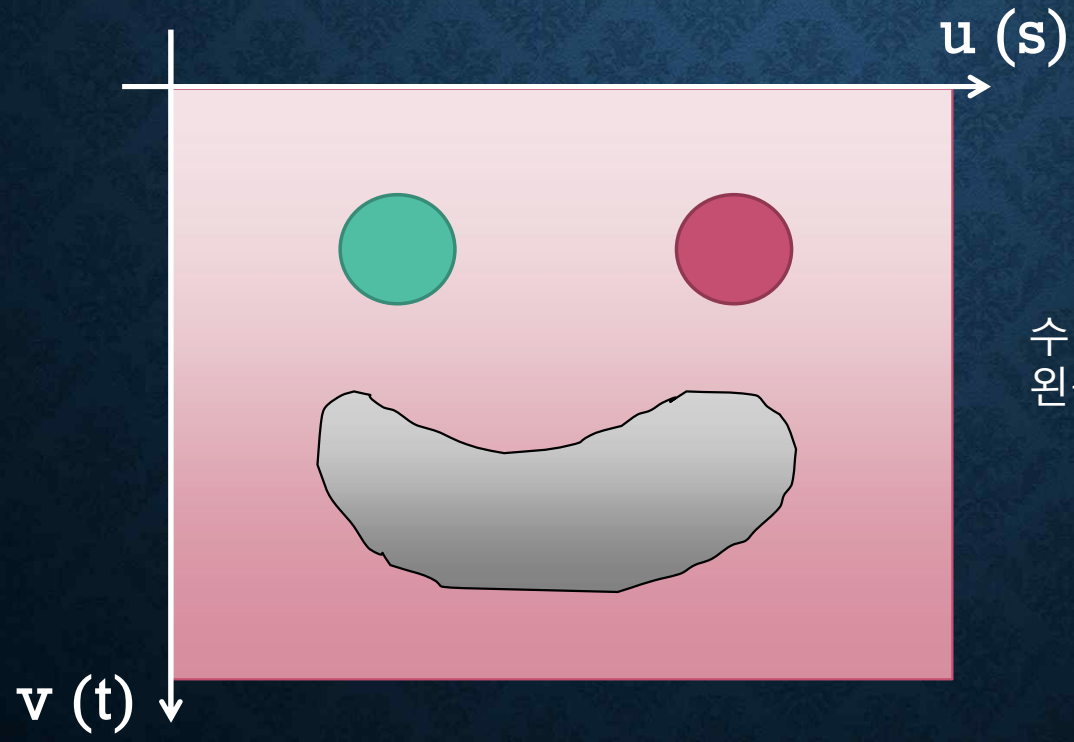
텍스쳐 매핑

- 필요한 데이터
  - 이미지 (jpg, bmp, png, tiff, …)
  - 매핑 위치



텍스쳐 매핑

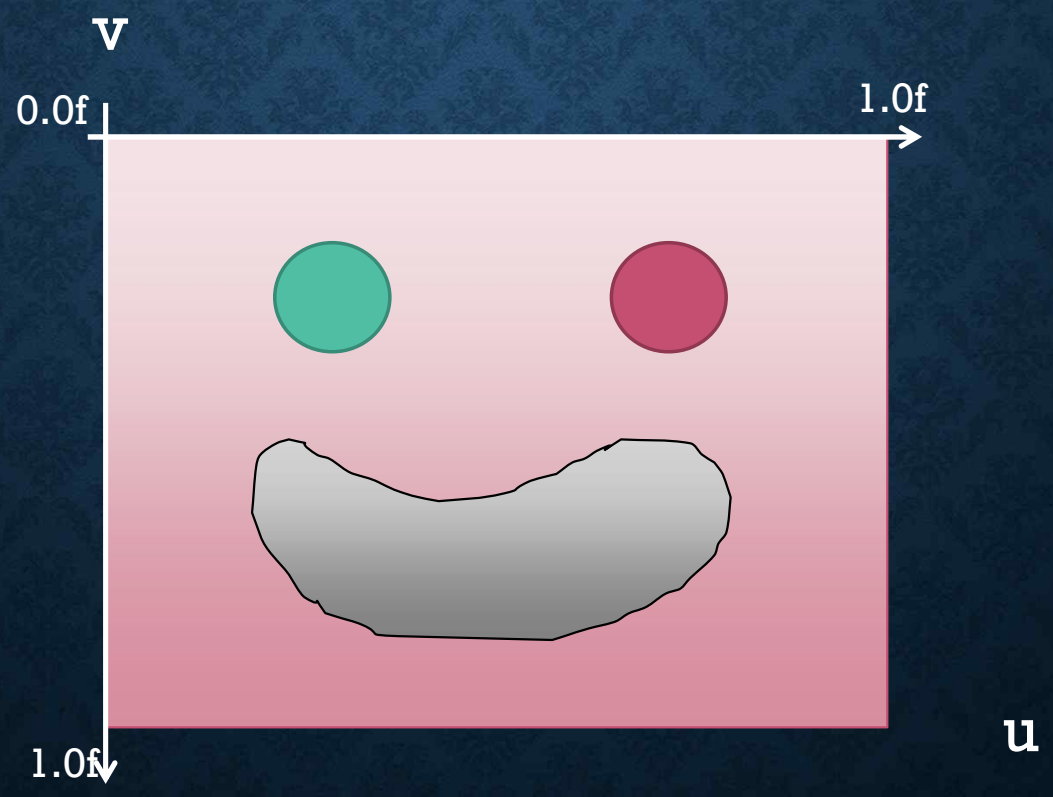
- 텍스쳐 상에서 사용되는 좌표계는 u, v 혹은 s, t 로 표현 됨



수업에선 u, v 의 축이
왼쪽 그림과 같다

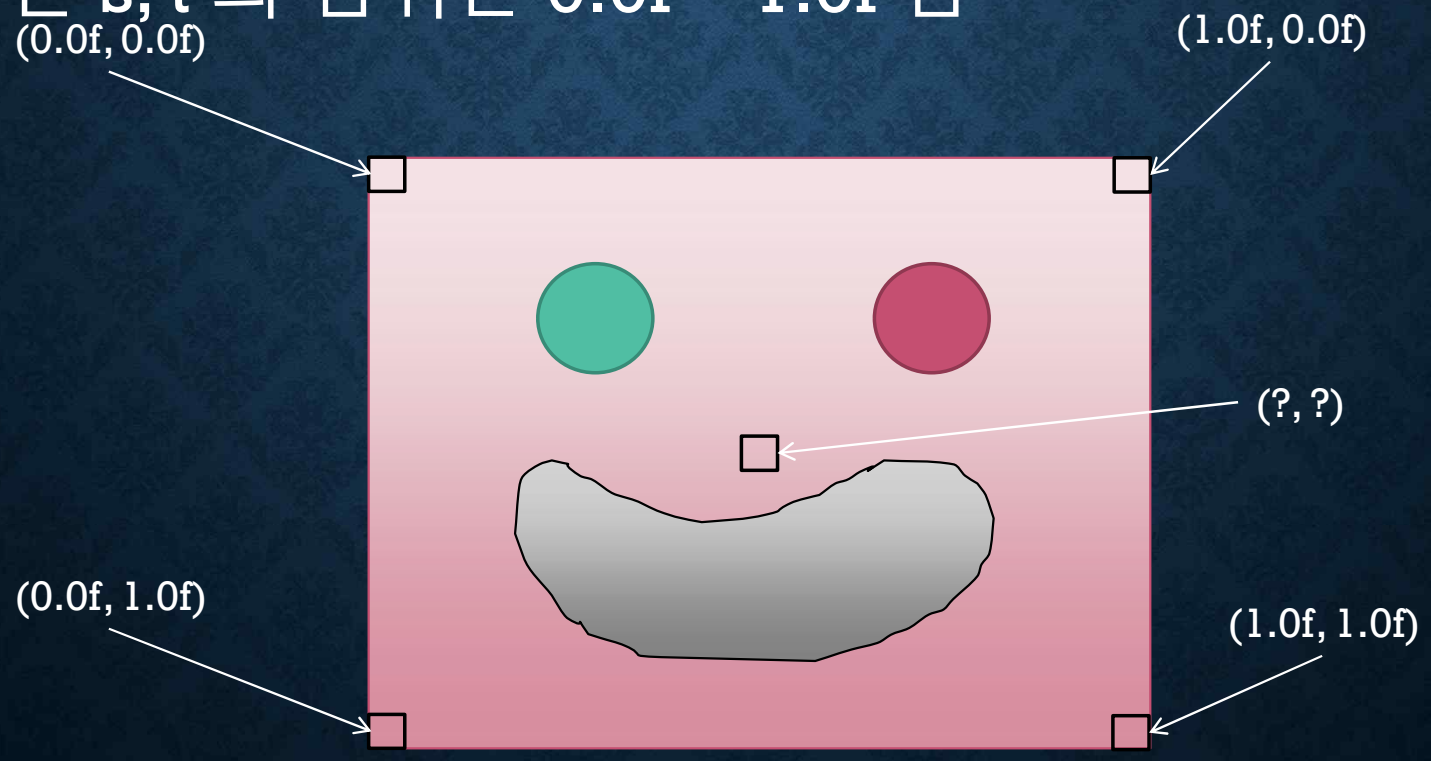
텍스쳐 매핑

- u, v 혹은 s, t 의 범위는 0.0f ~ 1.0f 임



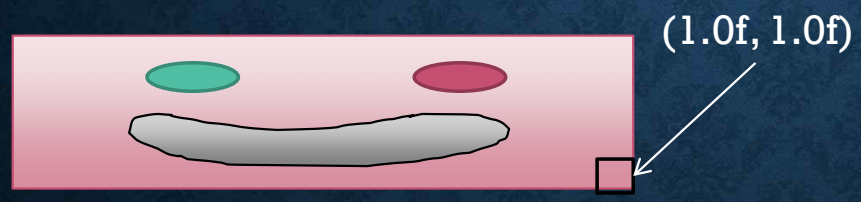
텍스쳐 매핑

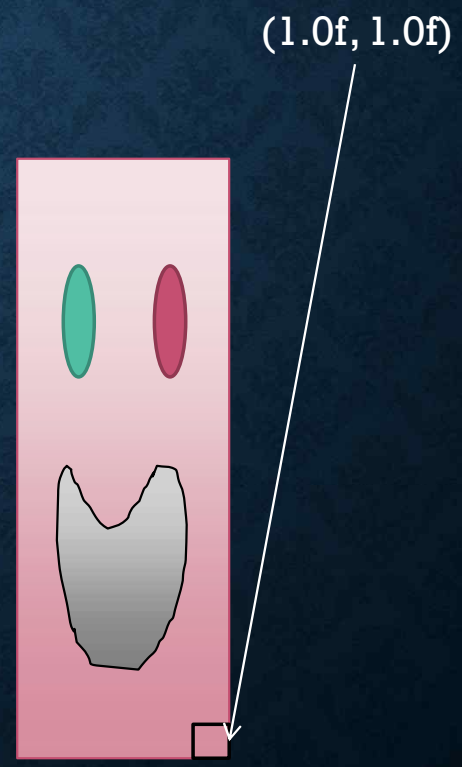
- u, v 혹은 s, t 의 범위는 0.0f ~ 1.0f 임



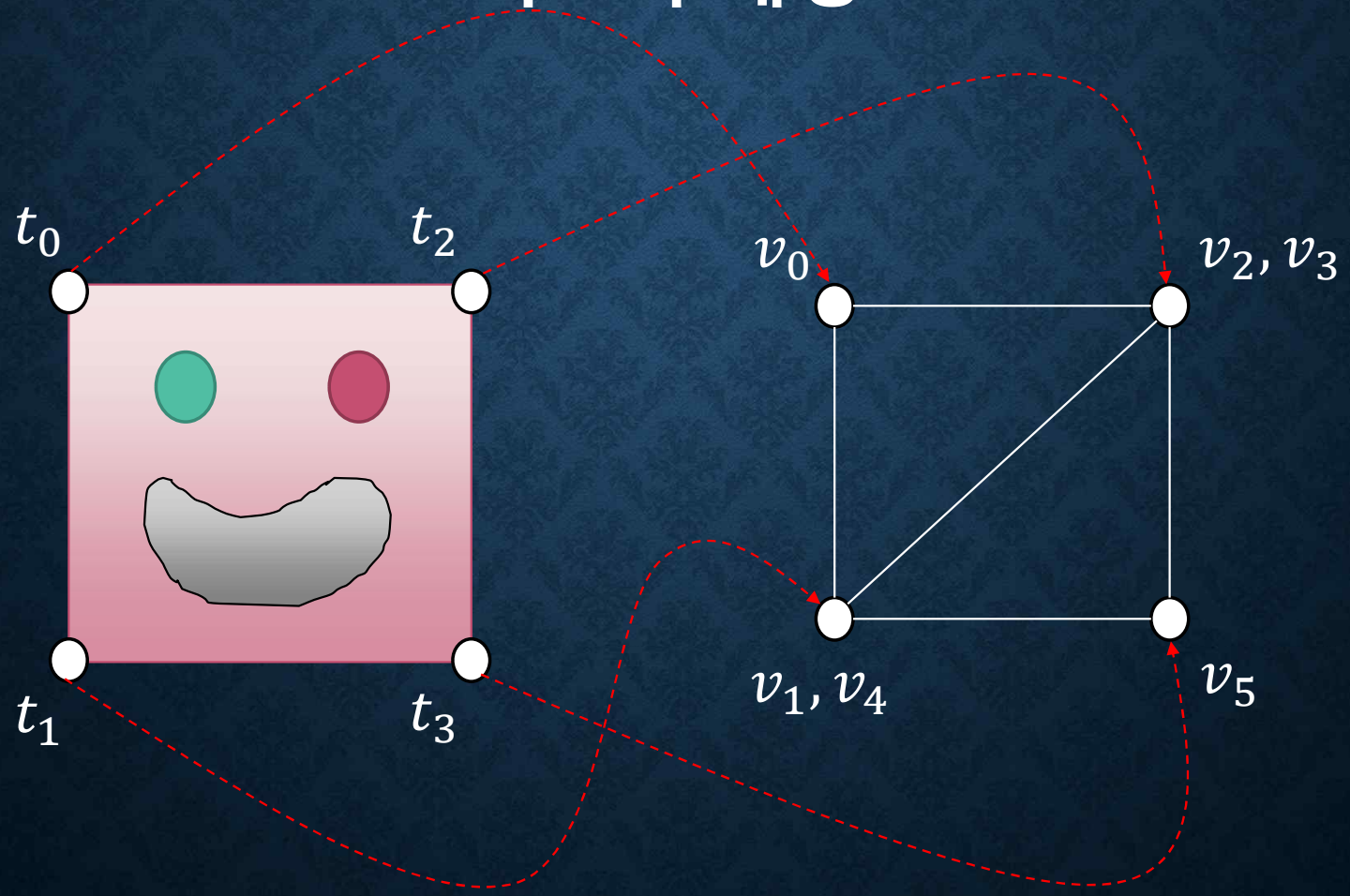
텍스쳐 매핑

- 텍스쳐 좌표는 0.0f ~ 1.0f 사이로 normalize 됨
- 이미지 해상도와 별개임

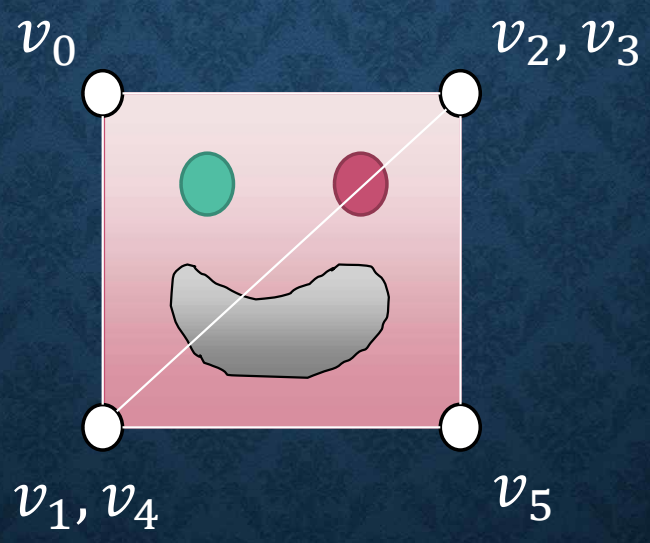




텍스쳐 매핑

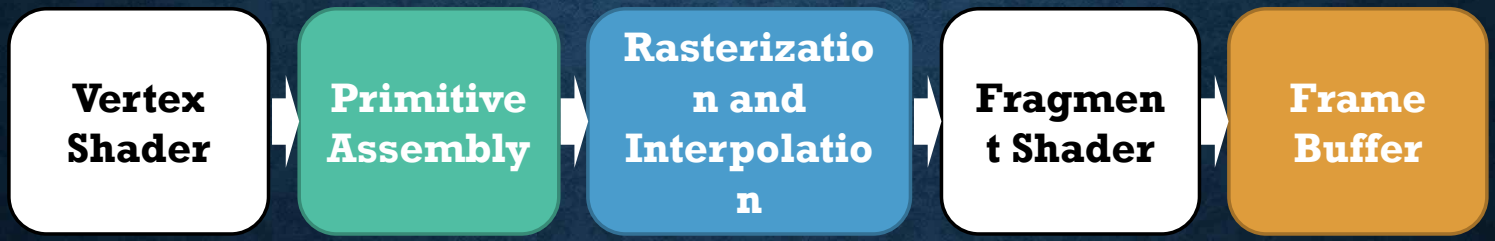


텍스쳐 매핑

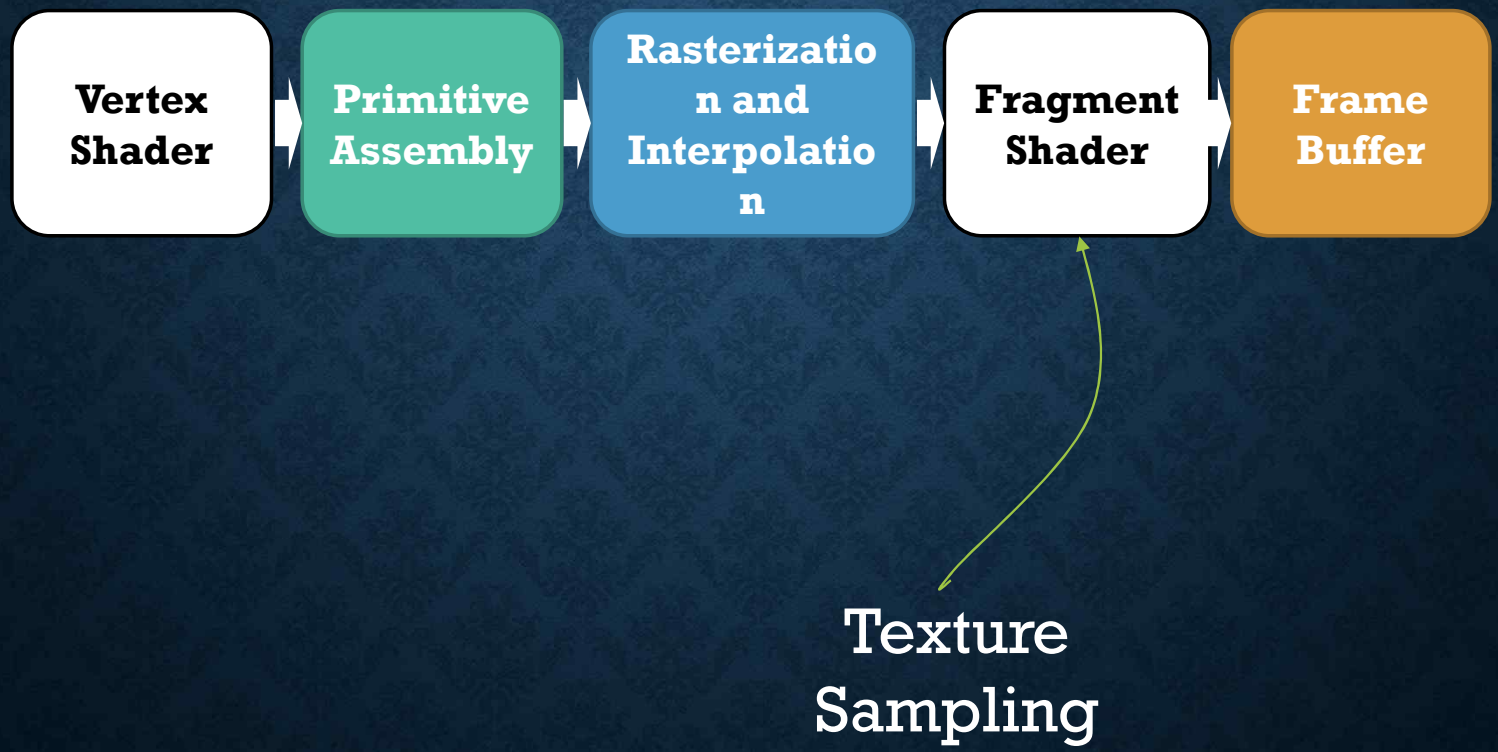


텍스쳐 매핑

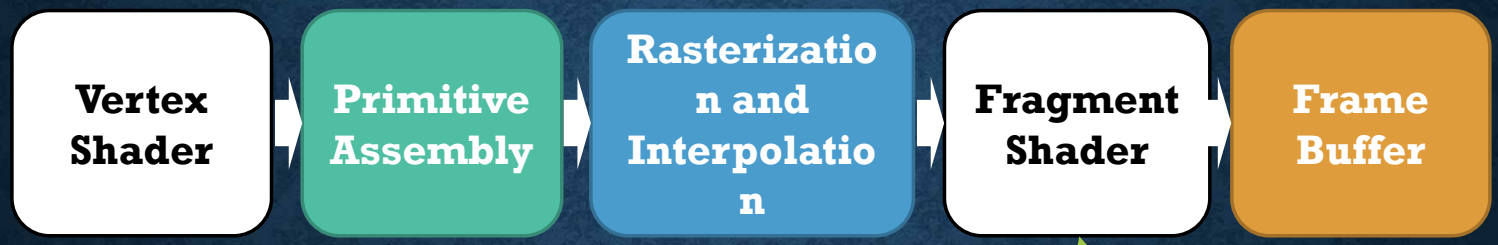
- 텍스쳐 샘플링은 어느 단계에서 이루어 질까?



텍스쳐 매핑


Vertex Shader
Primitive Assembly
Rasterization and Interpolation
Fragment Shader
Frame Buffer
Texture Sampling

텍스쳐 매핑

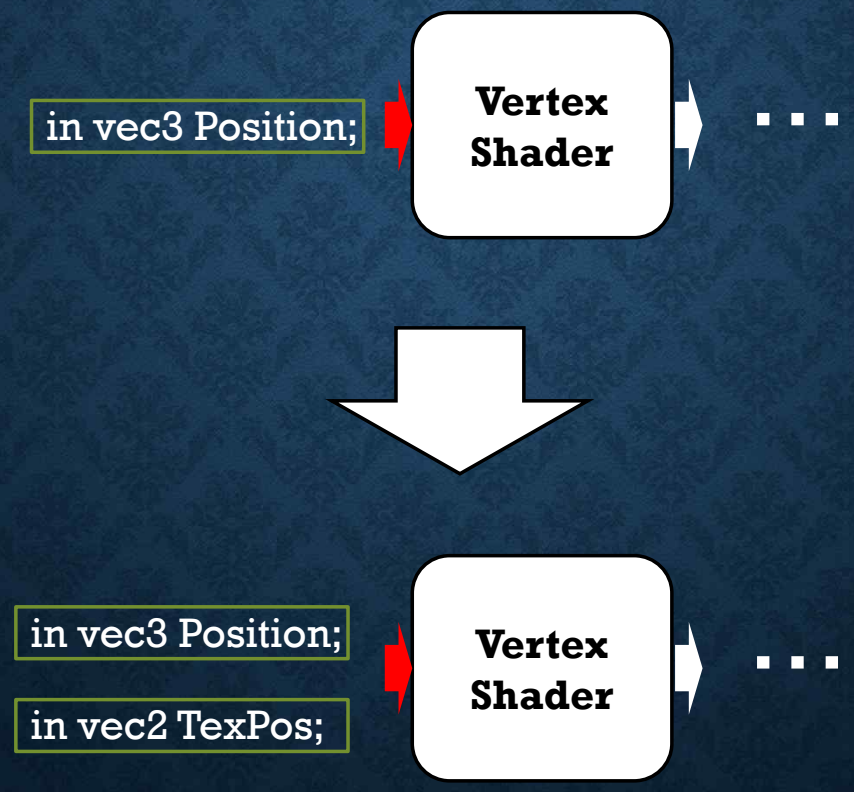


Sampling 을 위해선 텍스쳐 좌표가 필요하고 좌표를 얻기 위해선 버텍스 셰이더로 부터 받아야 함

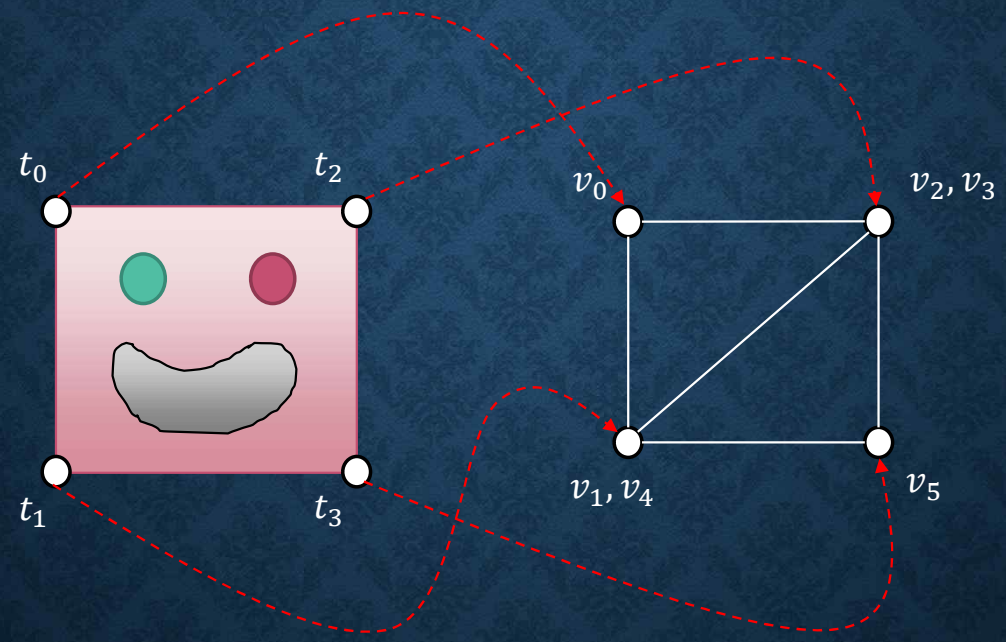
→ Attribute

Texture Sampling

텍스쳐 매핑

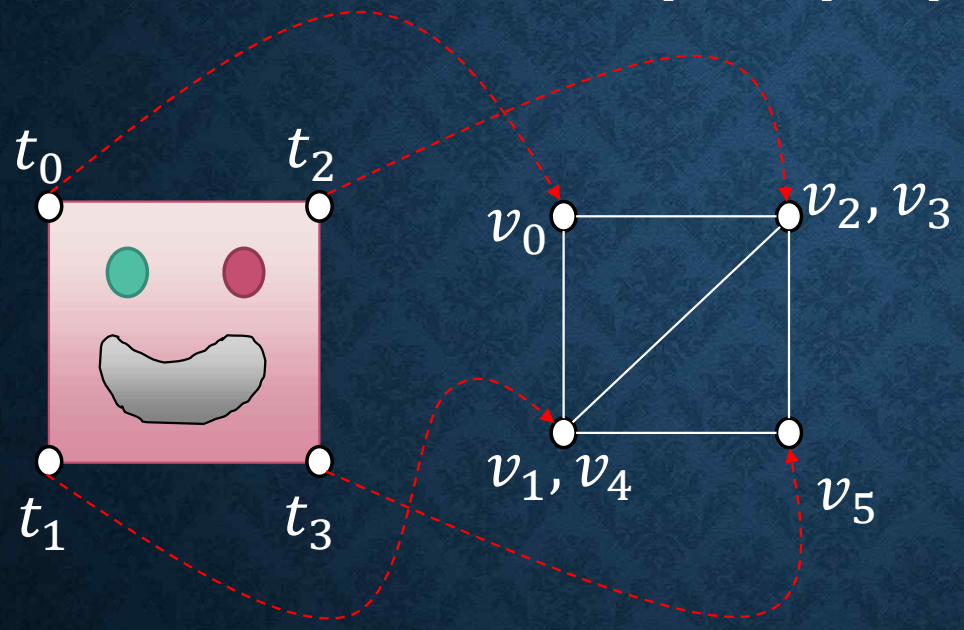

in vec3 Position;
Vertex Shader
...
in vec3 Position;
in vec2 TexPos;
Vertex Shader
...

텍스쳐 매핑



버텍스당 텍스쳐 좌표 하나씩 필요함
→ 버텍스 정보에 텍스쳐 좌표 추가 필요

텍스쳐 매핑

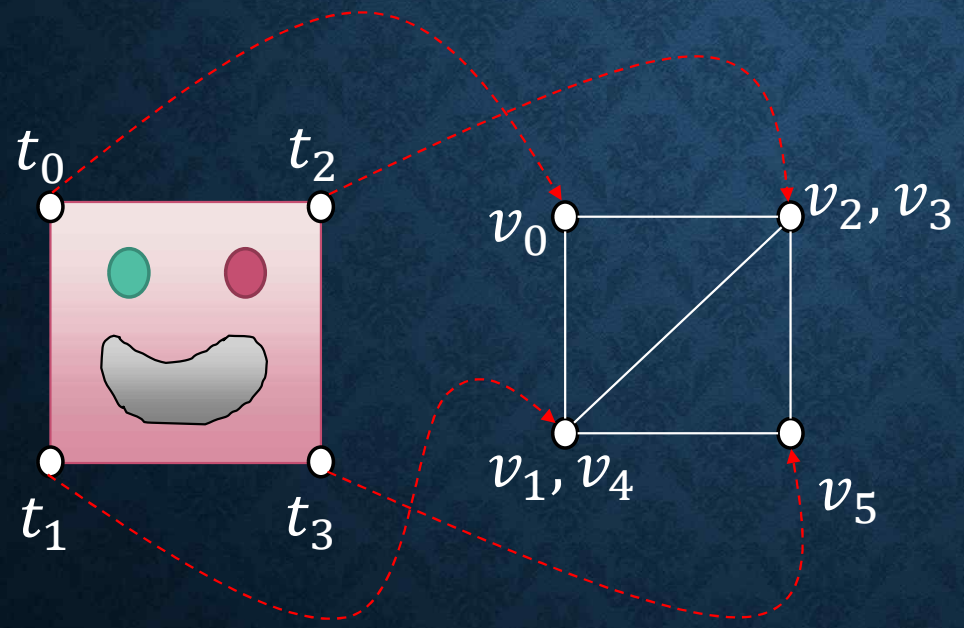


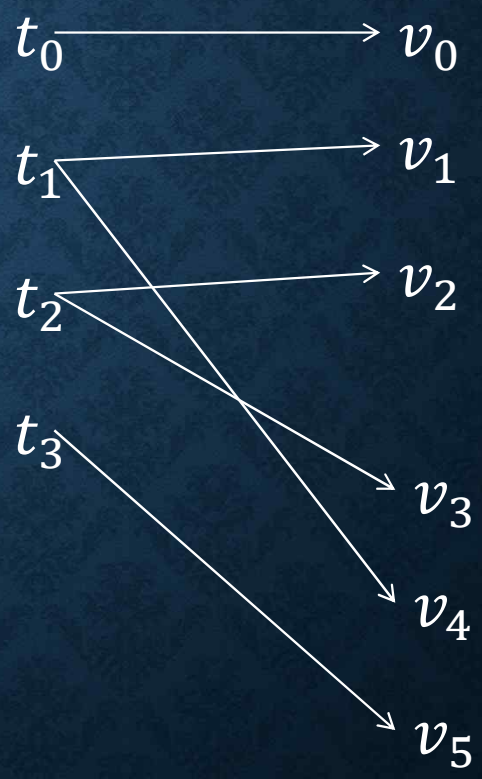
v_0 = (-0.5f, 0.5f, 0.0f)
v_1 = (-0.5f, -0.5f, 0.0f)
v_2 = (0.5f, 0.5f, 0.0f)

v_3 = (0.5f, 0.5f, 0.0f)
v_4 = (-0.5f, -0.5f, 0.0f)
v_5 = (0.5f, -0.5f, 0.0f)

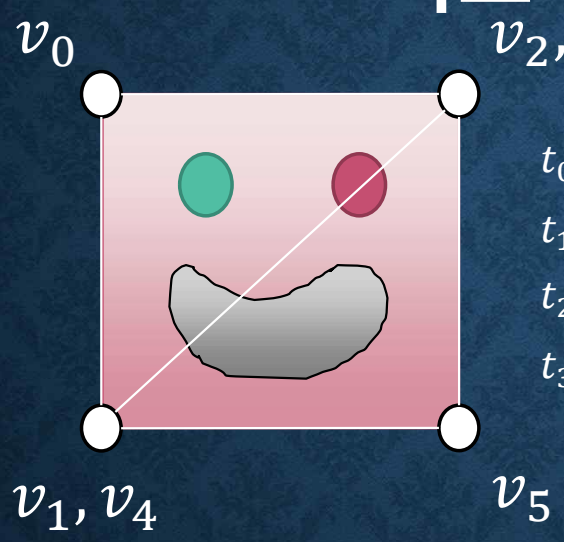
t_0 = (0.0f, 0.0f) t_2 = (1.0f, 0.0f)
t_1 = (0.0f, 1.0f) t_3 = (1.0f, 1.0f)

텍스쳐 매핑

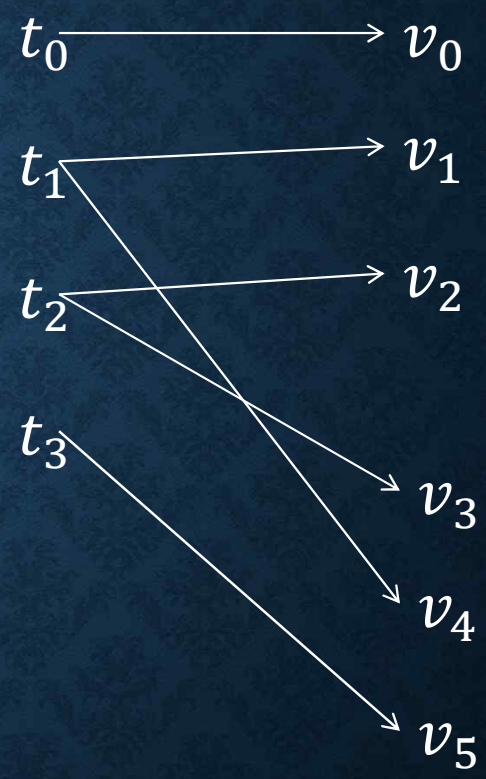




텍스쳐 매핑



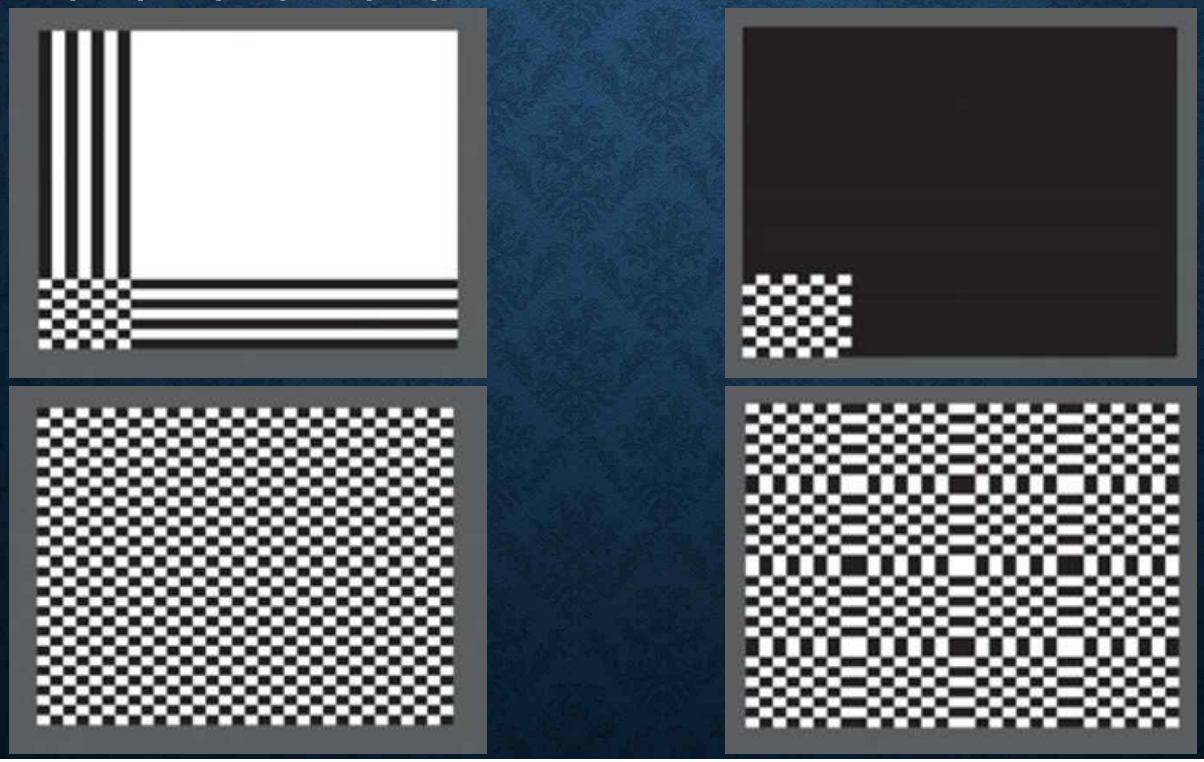
t_0 = (0.0f, 0.0f)
t_1 = (0.0f, 1.0f)
t_2 = (1.0f, 0.0f)
t_3 = (1.0f, 1.0f)



v_0 = (-0.5f, 0.5f, 0.0f, 0.0f, 0.0f)
v_1 = (-0.5f, -0.5f, 0.0f , 0.0f, 1.0f)
v_2 = (0.5f, 0.5f, 0.0f , 1.0f, 0.0f)
v_3 = (0.5f, 0.5f, 0.0f , 1.0f, 0.0f)
v_4 = (-0.5f, -0.5f, 0.0f , 0.0f, 1.0f)
v_5 = (0.5f, -0.5f, 0.0f , 1.0f, 1.0f)

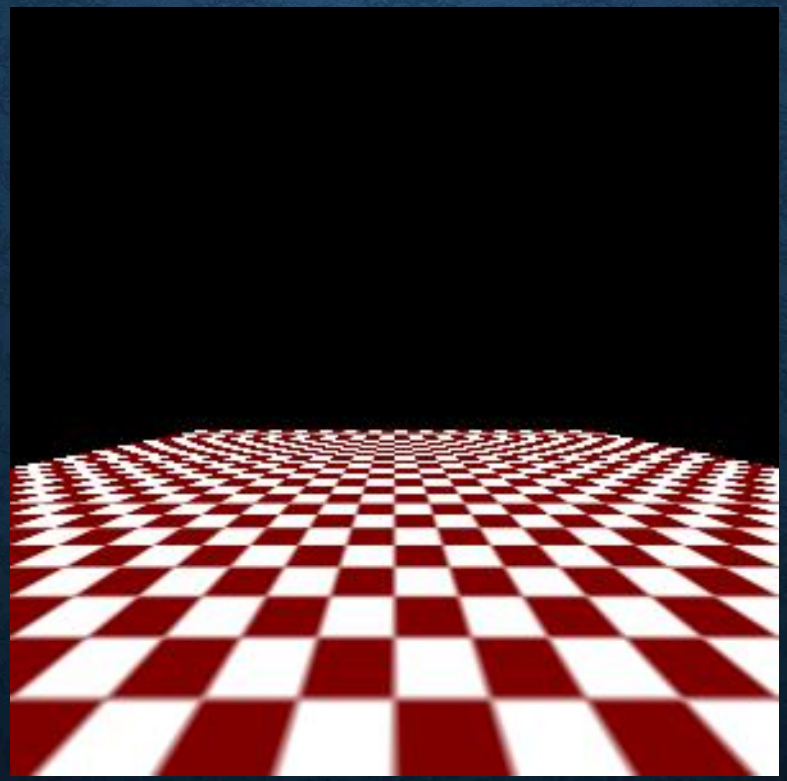
텍스쳐 매핑

- u, v 좌표가 0보다 작거나 1보다 클 경우
  - wrap 형식에 따라 채워짐



텍스쳐 매핑

- 텍스쳐 샘플링



텍스쳐 생성

참고 코드

```
GLuint CreatePngTexture(char* filePath, GLuint samplingMethod);
m_RGBTexture = CreatePngTexture("./rgb.png", GL_NEAREST);
```

```
#include "LoadPng.h"

GLuint Renderer::CreatePngTexture(char* filePath, GLuint samplingMethod)
{
	//Load Png
	std::vector<unsigned char> image;
	unsigned width, height;
	unsigned error = lodepng::decode(image, width, height, filePath);
	if (error != 0)
	{
		std::cout << "PNG image loading failed:" << filePath << std::endl;
		assert(0);
	}
```

텍스쳐 사용

텍스쳐 사용

```
#version 330

in vec3 Position;
in vec2 TexPos;

out vec2 vTexPos;

void main()
{
    gl_Position = vec4(Position, 1.0);
    vTexPos = TexPos;
}
```

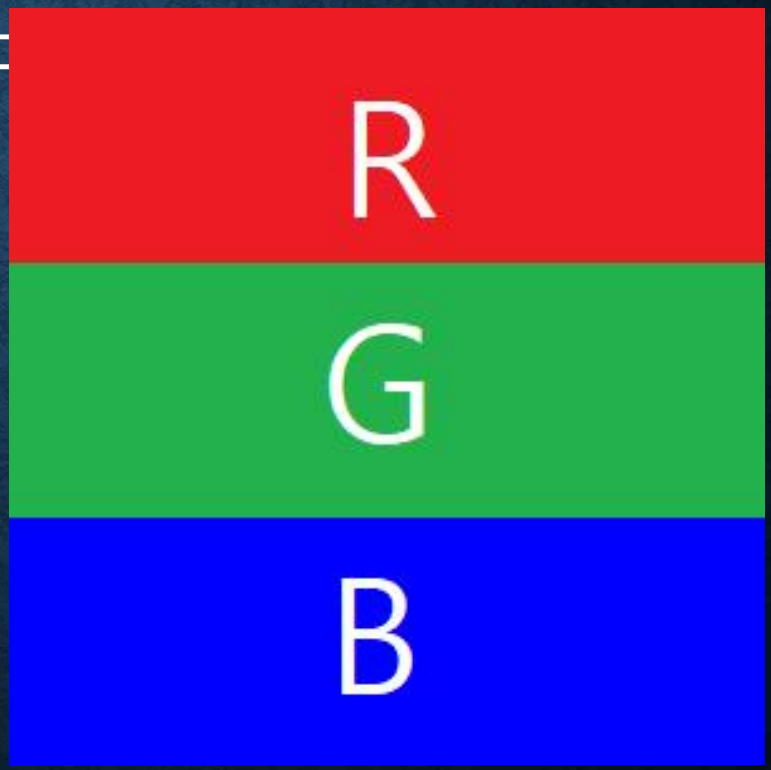
```
#version 330

uniform sampler2D uTexSampler;
in vec2 vTexPos;

out vec4 FragColor;

void main()
{
    FragColor = texture(uTexSampler, vTexPos);
}
```

```
int uniformTex = glGetUniformLocation(gShaderProgram, "uTexSampler");
glUniform1i(uniformTex, 0);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, gTextureID);
```

실습

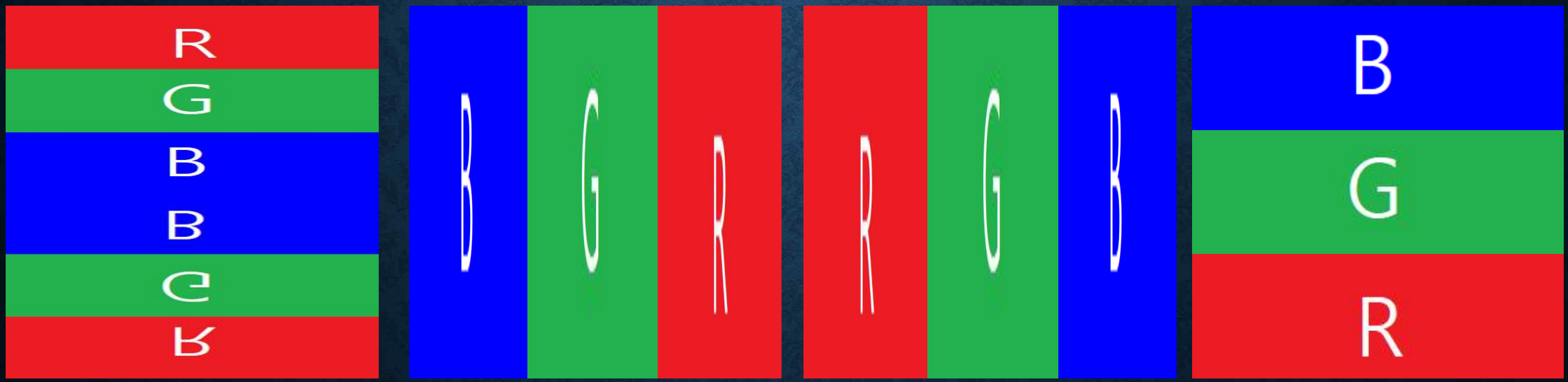
실습

- 새로운 Shader 및 API 생성 후 ㄱ
- 문제 : 블러 효과 구현 해보기
- 문제 : 좌표 꼬아 보기



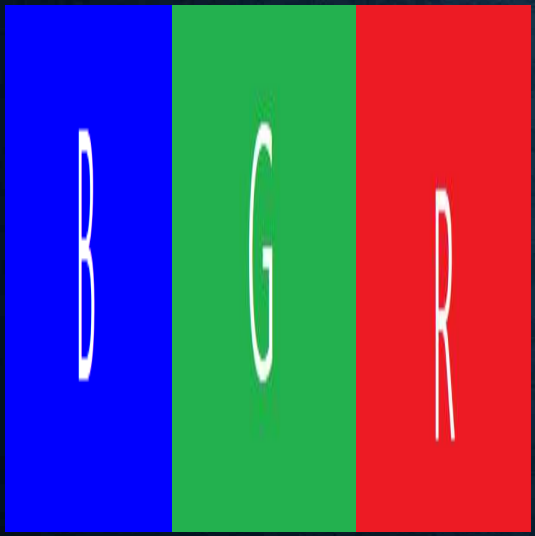
실습

문제 : 좌표 꼬아 보기

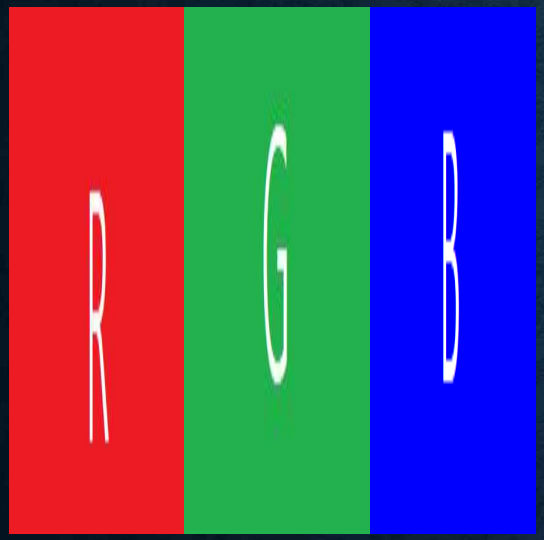


R
G
B

R
G
B
B
G
R


B
G
R

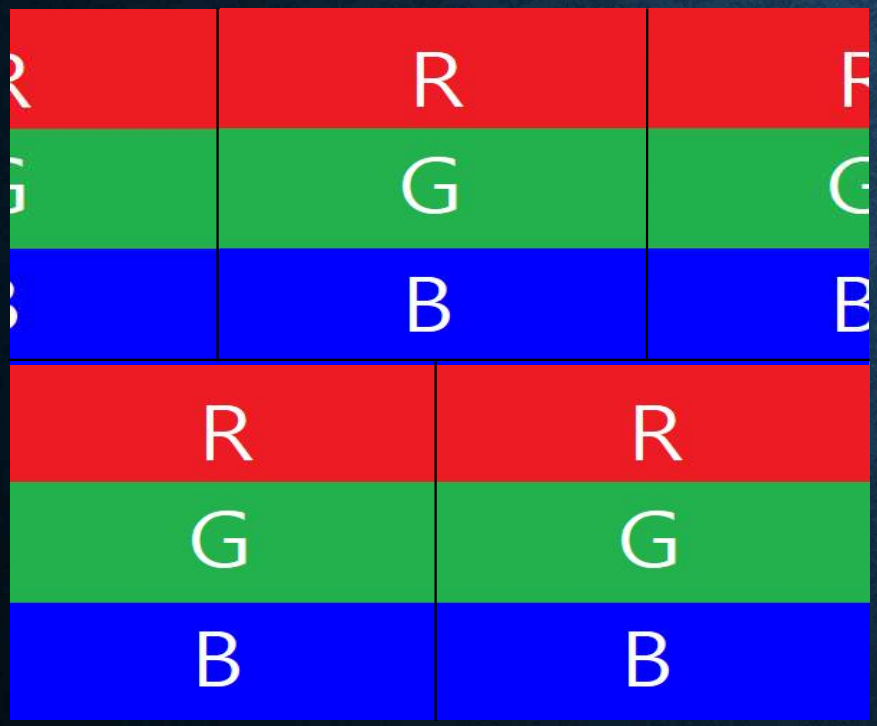
R
G
B


R
G
B

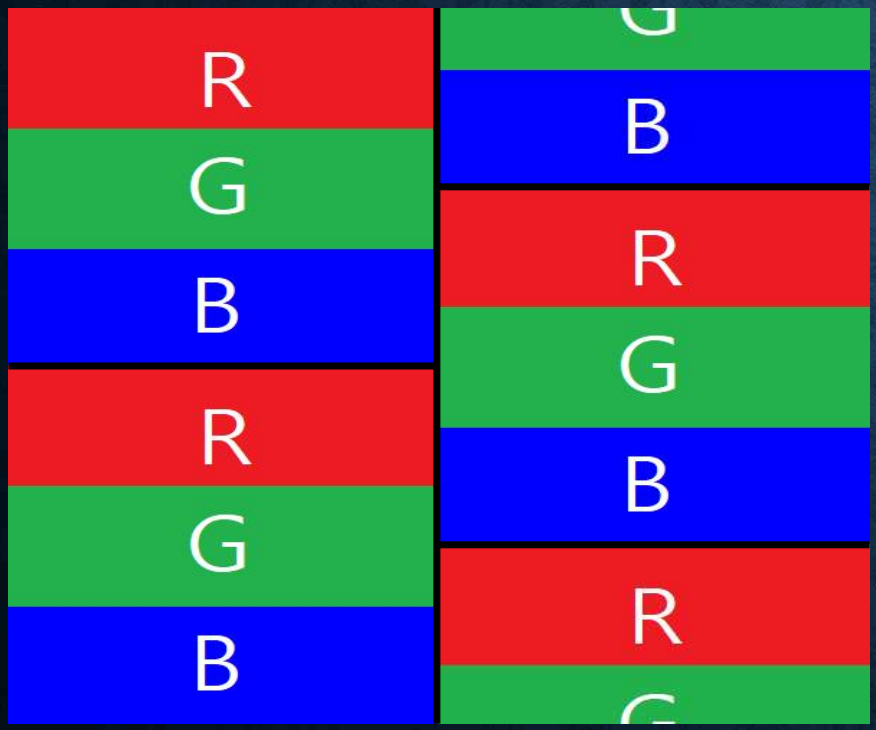
R
G
B

R
G
B

실습



실습



다중 텍스처 사용

다중 텍스처 사용

- 지금까지는 Texture 하나만 사용
  - Default 값이 0 으로 지정되어 있기 때문에 따로 지정을 해 주지 않아도 동작함

Fragment Shader

```
#version 330

in vec2 vTexPos;

out vec4 FragColor;

uniform sampler2D uTexture;

void main()
{
   FragColor = texture(uTexture, vTexPos);
}
```

```
int uniformTex = glGetUniformLocation(gShaderProgram, "uTexSampler");

glUniform1i(uniformTex, 0);

glActiveTexture(GL_TEXTURE0);

glBindTexture(GL_TEXTURE_2D, gTextureID);
```

ActiveTexture 가 없어도 동작함

다중 텍스처 사용

- Texture 가 여러 장일 경우 glActiveTexture 함수를 통해 사용할 Texture 의 번호를 지정해 주어야 함

gTextureID → GL_TEXTURE0
gTextureID1 → GL_TEXTURE1
gTextureID2 → GL_TEXTURE2
gTextureID3 → GL_TEXTURE3
…

다중 텍스처 사용

gTextureID → GL_TEXTURE0 지정 방법

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, gTextureID);
```

gTextureID → GL_TEXTURE1 지정 방법

```
glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, gTextureID1);
```

다중 텍스처 사용

- 즉, GL_TEXTURE_2D 는 설정 가능한 텍스처가 최소 80개가 있음
- 각 텍스처를 설정하기 위해 Bind 전에 Active 시키는 과정이 필요함

1. Active

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, gTextureID);
```

2. Bind

다중 텍스처 사용

GL_TEXTURE_2D

- GL_TEXTURE0 — glActiveTexture(GL_TEXTURE0) → glBindTexture(GL_TEXTURE_2D, gTextureID);
- GL_TEXTURE1 — glActiveTexture(GL_TEXTURE1) → glBindTexture(GL_TEXTURE_2D, gTextureID1);
- GL_TEXTURE2 — glActiveTexture(GL_TEXTURE2) → glBindTexture(GL_TEXTURE_2D, gTextureID2);
- GL_TEXTURE3 — glActiveTexture(GL_TEXTURE3) → glBindTexture(GL_TEXTURE_2D, gTextureID3);
- GL_TEXTURE4 — glActiveTexture(GL_TEXTURE4) → glBindTexture(GL_TEXTURE_2D, gTextureID4);
- GL_TEXTURE5 — glActiveTexture(GL_TEXTURE4) → glBindTexture(GL_TEXTURE_2D, gTextureID5);
- GL_TEXTURE6
- …
- …
- …
- …
- …
- …
- …
- …
- …
- GL_TEXTURE31

다중 텍스처 사용

GL_TEXTURE_2D

- GL_TEXTURE0 : gTextureID
- GL_TEXTURE1 : gTextureID1
- GL_TEXTURE2 : gTextureID2
- GL_TEXTURE3 : gTextureID3
- GL_TEXTURE4 : gTextureID4
- GL_TEXTURE5 : gTextureID5
- GL_TEXTURE6 : nothing
- …
- …
- …
- …
- …
- …
- …
- …
- …
- GL_TEXTURE31 : nothing

다중 텍스처 사용(실습 준비)

Vertex Shader

```
#version 330

in vec3 Position;
in vec2 TexPos;

out vec2 vTexPos;

void main()
{
   gl_Position = vec4(Position, 1.0);
   vTexPos = TexPos;
}
```

Fragment Shader

```
#version 330

in vec2 vTexPos;

out vec4 FragColor;

uniform sampler2D uTexSampler;

void main()
{
   FragColor = texture(uTexSampler, vTexPos);
}
```

다중 텍스처 사용(실습 준비)

```
float vertPosTex[30] =
{
-0.5f, 0.5f, 0.0f, 0.0f, 1.0f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.5f, 0.5f, 0.0f, 1.0f, 1.0f,
0.5f, 0.5f, 0.0f, 1.0f, 1.0f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f
};

glGenBuffers(1, &VBO_PosTex);
glBindBuffer(GL_ARRAY_BUFFER, VBO_PosTex);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertPosTex), vertPosTex, GL_STATIC_DRAW);
```

다중 텍스처 사용(실습 준비)

```
glUseProgram(gShaderProgram);

glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, gTextureID);

glActiveTexture(GL_TEXTURE1);
glBindTexture(GL_TEXTURE_2D, gTextureID1);

glActiveTexture(GL_TEXTURE2);
glBindTexture(GL_TEXTURE_2D, gTextureID2);

glActiveTexture(GL_TEXTURE3);
glBindTexture(GL_TEXTURE_2D, gTextureID3);

glActiveTexture(GL_TEXTURE4);
glBindTexture(GL_TEXTURE_2D, gTextureID4);

glActiveTexture(GL_TEXTURE5);
glBindTexture(GL_TEXTURE_2D, gTextureID5);
```

다중 텍스처 사용(실습 준비)

```
int uniformTex = glGetUniformLocation(gShaderProgram, "uTexSampler");
glUniform1i(uniformTex, 0);
```

다중 텍스처 사용(실습 준비)

```
int attrribPosition = glGetAttribLocation(gShaderProgram, "Position");
int attrribTexPos = glGetAttribLocation(gShaderProgram, "TexPos");

glEnableVertexAttribArray(attrribPosition);
glEnableVertexAttribArray(attrribTexPos);

glBindBuffer(GL_ARRAY_BUFFER, VBO_PosTex);
glVertexAttribPointer(attrribPosition, 3, GL_FLOAT, GL_FALSE, 5 * sizeof(float), 0);
glVertexAttribPointer(attrribTexPos, 2, GL_FLOAT, GL_FALSE, 5 * sizeof(float), (GLvoid*)(3 * sizeof(float)));
```

다중 텍스처 사용(실습 준비)

```
glDrawArrays(GL_TRIANGLES, 0, 6);
```

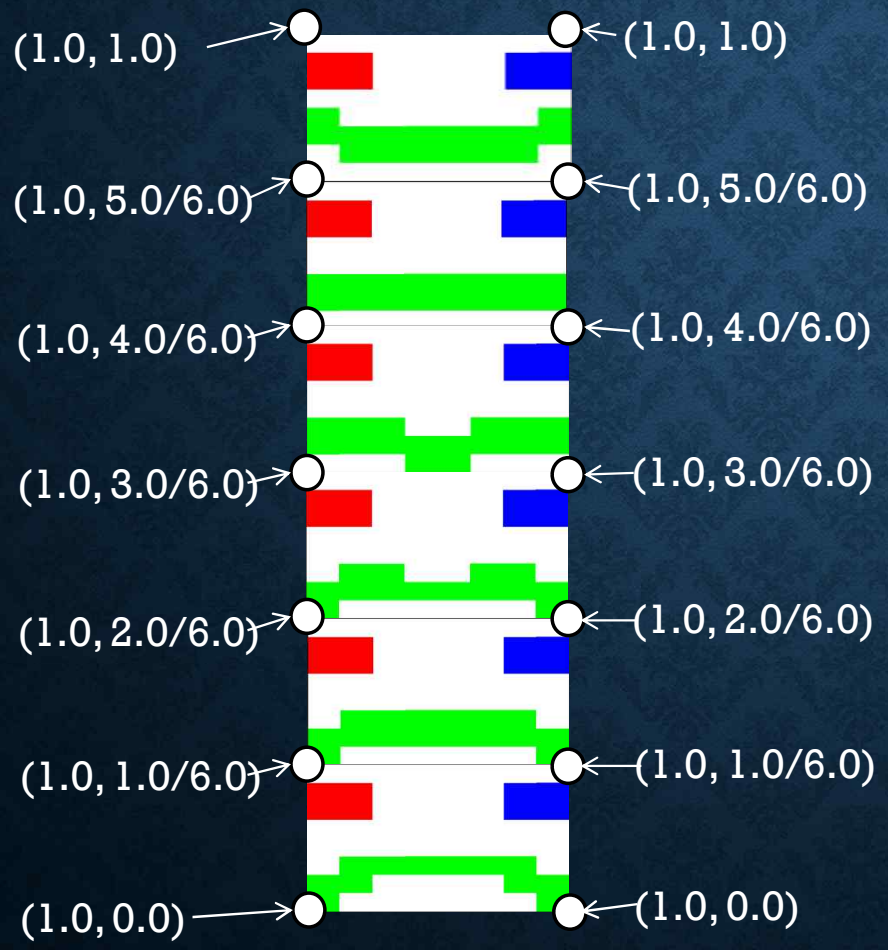
실습

- 내용 구현
- uniform 값(sampler2D) 을 지속적으로 변경하여 gTextureID0 ~ gTextureID5 까지 애니메이션 되도록 구현
  - 너무 빠르게 바뀐다면 Sleep(1000); 을 넣어서 천천히 확인할 수 있음

단일 텍스처 사용 애니메이션

- 다중 텍스처를 사용하여 애니메이션 구현
  - 텍스처 Switch 가 발생하여 효율성이 떨어짐
  - 캐시 효율성이 떨어짐
- 하나의 텍스처에 여러장의 텍스처를 합쳐서 그리는 방식
  - 텍스처 Switch 발생이 없음
  - 캐시 효율성 높음

단일 텍스처 사용 애니메이션



numbers.png

1	2	3	4	5
6	7	8	9	0

35%